The Journal of Open Source Software

optrace: A Python library for 3D optical raytracing and polychromatic image synthesis

Damian Mendroch [1]

1 Institute for Applied Optics and Electronics, TH Köln – University of Applied Sciences, Cologne, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 14 July 2026
Published: unpublished

Summary

Alongside experimental setups, simulation is a key component of optical design and analysis. Physics-based modeling tools allow for the approximation of real-world optical problems with significantly greater flexibility, cost-efficiency, and versatility compared to classical prototyping and measurement. The engineering aspect of such tools places a strong emphasis on optimizing setups in terms of aberrations, cost, and feasibility. In addition, another key aspect is the demonstration and explanation of optical effects: This is not only relevant for educational purposes but also for an application-oriented visualization of system properties. Examples include realistically rendered images through camera lenses or simulations of a patient's vision with an artificial intraocular lens. In this context, there is a gap in available open-source tools that this work aims to address.

For this purpose, we provide a package that enables the simulation of color images through raytracing, even for more complex setups, and includes analysis tools such as a 3D viewer, paraxial calculation tools, and automated scripting.

Statement of need

optrace offers the following features:

- Sequential raytracing of freely definable geometries
- Rendering of grayscale or color images on detector surfaces
- Iterative search for the focal position
- Alternative image simulation mode by PSF convolution
- Paraxial analysis involving focal positions, image distances, and pupils
- User-definable spectra, refractive indices, geometries, surfaces, simulation images, and a variety of presets
- Import of basic .zmx Zemax geometry files and .agf material catalogues
- Optional edge diffraction approximation using Heisenberg uncertainty ray bending (Heinisch & Chou, 1971)

The presented tool has multiple use cases: One application is the educational demonstration of aberrations and fundamental optical configurations. Here, it serves as an introductory framework for paraxial optics, geometric optics, and image formation. Furthermore, the software enables the simulation of schematic optical systems, such as eye models, prisms, and telescopes. With these features, it offers an accessible alternative to professional-grade optical design suites for preliminary system estimations.

State of the field

There is a wide variety of open-source projects available for the optics domain. While these tools are more constrained compared to the proprietary suites such as Ansys Zemax OpticStudio, Lambda Research Corporation OSLO, and Quadoa by Quadoa Optical Systems, they can provide targeted and easily accessible simulation capabilities. Geometric optical libraries, focusing on ray-based analysis, include rayopt (QUARTIQ, 2020), Optiland (Harrison, 2026), RayOptics (Hayford, 2025), RayTracing (DCC-Lab, 2026), and tracepy (Niendorf, 2025). Purely wave-optical simulations are facilitated by frameworks such as diffractsim (Fuente Herrezuelo, 2022), poppy (Perrin et al., 2016), and prysm (Dube, 2019). Furthermore, hybrid architectures that integrate both physical principles are provided by opticspy (Fan, 2015), raypier (Cole, 2021), and PAOS (Bocchieri et al., 2024).

Despite the availability of these projects, there remains a notable gap regarding the simulation of realistic, polychromatic imagery within a 3D optics environment featuring custom-defined surfaces. Furthermore, no existing package is known to consolidate paraxial analysis, sequential raytracing, and PSF-based image simulation with an interactive graphical user interface and 3D visualization within a unified framework.

Software design

The primary design decision involves the definition of the functional scope and its inherent constraints. Central to this framework is the simulation of geometric optics via sequential raytracing. Consequently, wave-optical modeling and non-sequential configurations are excluded. Physical phenomena such as diffraction, interference, and reflections are beyond the current simulation capabilities. Furthermore, `optrace` does not provide tools for aberration analysis or automated geometry optimization. This specialized functional focus reduces the systemic complexity and allows for prioritizing the computational efficiency of the implemented methods.

In contrast to the GUI-centric workflows of various commercial applications, geometry definition and simulation follow a scripting-based approach. The graphical user interface is entirely optional, facilitating a high degree of automation, modularity, and seamless integration with external libraries and existing codebases. Regarding internal modularity, the core tracing functionality is included within the `optrace` namespace, distinct from the plotting (`optrace.plots`) and GUI (`optrace.gui`) modules. This decoupled architecture ensures that components remain optional, interchangeable, and easily extensible.

Significant emphasis is placed on usability and a low barrier to entry. To this end, the library is supported by comprehensive online documentation and an extensive suite of practical examples. Furthermore, it includes a wide variety of presets for images, spectra, refractive indices, and surface geometries. Next, the Python programming language was selected due to its widespread adoption in the scientific community and its vast ecosystem of specialized libraries.

To maintain high performance despite the high-level nature of the language, computationally intensive components are offloaded to optimized numerical libraries, including NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), and OpenCV (Bradski, 2000). Performance is further enhanced through the implementation of multithreading for resource-heavy operations and memory-efficient data structures for ray parameter management. Additionally, analytical solutions are prioritized over numerical or iterative methods to ensure both precision and speed.

To maintain a specialized functional scope, the library delegates a multitude of tasks to established external libraries. 2D-visualization is handled by matplotlib (Hunter, 2007), while 3D rendering and spatial data representation leverage VTK (Schroeder et al., 2006) and pyvista (Sullivan & Kaszynski, 2019). User interface related libraries comprise PySide (The Qt Company, 2026), pyvistaqt (pyvista, 2026), TraitsUI (Enthought, Inc., 2023), and pyqtdarktheme-fork (Ohsugi & Lima, 2026). Additional dependencies include chardet (chardet, 2026) for character

encoding detection and tqdm (Costa-Luis et al., 2026) for progress visualization.

The preceding descriptions provide only a concise overview of the library. Comprehensive application examples are available in the Examples section of the online documentation, complemented by a detailed User Guide. Furthermore, extensive resources regarding the API and the underlying mathematical and technical implementation details are provided for further reference.

Research impact statement

Research utilizing `optrace` has already resulted in two peer-reviewed publications for the simulation of intraocular lenses (IOLs) (Mendroch et al., 2024, 2025). The first work focused on the optical characterization of the IOL by analyzing the refractive power and raytracing behavior. The second work expanded this scope by integrating these lenses in a full-eye model and generating realistic polychromatic retinal images.

Beyond research, the software is actively utilized at the Institute of Applied Optics and Electronics, TH Köln – University of Applied Sciences, Germany, to create teaching materials and perform basic simulation tasks.

By providing access to this versatile optics tool, we aim to support the broader academic community and enhance both research and educational applications in the field of optics.

AI usage disclosure

No generative AI tools were used in the development of this software. AI tools were solely used for proofreading, linguistic refinement and polishing of the project documentation and this paper. All produced content has been reviewed by a human reader.

Acknowledgements

The author would like to thank Prof. Dr. Stefan Altmeyer and Prof. Dr. Uwe Oberheide from the Institute of Applied Optics and Electronics at TH Köln – University of Applied Sciences for providing the original inspiration and valuable conceptual guidance for this project.

References

Bocchieri, A., Mugnai, L. V., & Pascale, E. (2024). PAOS: A fast, modern, and reliable python package for physical optics studies. In L. E. Coyle, M. D. Perrin, & S. Matsuura (Eds.), *Space telescopes and instrumentation 2024: Optical, infrared, and millimeter wave* (p. 290). SPIE. https://doi.org/10.1117/12.3018333

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

chardet. (2026). *Chardet – universal character encoding detector*. https://github.com/chardet/chardet

Cole, B. (2021). *Raypier: A non-sequential optical ray-tracing framework*. https://github.com/bryancole/raypier_optics

Costa-Luis, C. da, Larroque, S. K., Altendorf, K., Mary, H., richardsheridan, Korobov, M., Raphael, N., Ivanov, I., Bargull, M., Rodrigues, N., Shawn, Dektiarev, M., Górny, M., mjstevens777, Pagel, M. D., Zugnoni, M., CrazyPython, Newey, C., Lee, A., … Kemenade, H. van. (2026). *Tqdm: A fast, extensible progress bar for python and CLI* (Version v4.67.3). Zenodo. https://doi.org/10.5281/zenodo.18473238

DCC-Lab. (2026). *RayTracing*. https://github.com/DCC-Lab/RayTracing

Dube, B. (2019). Prysm: A python optics module. *Journal of Open Source Software*, *4*(37), 1352. https://doi.org/10.21105/joss.01352

Enthought, Inc. (2023). *TraitsUI: Traits-capable windowing framework*. https://github.com/enthought/traitsui/

Fan, X. (2015). *Opticspy*. https://github.com/Sterncat/opticspy

Fuente Herrezuelo, R. de la. (2022). *Diffractsim: A flexible python diffraction simulator*. GitHub. https://doi.org/10.5281/zenodo.6147771

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Harrison, K. (2026). *Optiland*. https://github.com/optiland/optiland

Hayford, M. J. (2025). *RayOptics*. https://github.com/mjhoptics/ray-optics

Heinisch, R. P., & Chou, T. S. (1971). Numerical experiments in modeling diffraction phenomena. *Applied Optics*, *10*(10), 2248. https://doi.org/10.1364/ao.10.002248

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Mendroch, D., Altmeyer, S., & Oberheide, U. (2025). Polychromatic virtual retinal imaging of two extended-depth-of-focus intraocular lenses. *Translational Vision Science & Technology*, *14*(12), 33. https://doi.org/10.1167/tvst.14.12.33

Mendroch, D., Oberheide, U., & Altmeyer, S. (2024). Functional design analysis of two current extended-depth-of-focus intraocular lenses. *Translational Vision Science & Technology*, *13*(8), 33. https://doi.org/10.1167/tvst.13.8.33

Niendorf, G. (2025). *Tracepy*. https://github.com/TracePy-Org/tracepy

Ohsugi, Y., & Lima, H. G. P. (2026). *PyQtDarkTheme-fork*. https://github.com/henriquegemignani/PyQtDarkTheme

Perrin, M., Long, J., Douglas, E., Sivaramakrishnan, A., Slocum, C., & others. (2016). *POPPY: Physical optics propagation in PYthon*. Astrophysics Source Code Library. https://ascl.net/1602.018

pyvista. (2026). *PyVistaQt*. https://github.com/pyvista/pyvistaqt

QUARTIQ. (2020). *RayOpt*. https://github.com/quartiq/rayopt

Schroeder, W., Martin, K., & Lorensen, B. (2006). *The visualization toolkit (4th ed.)*. Kitware. ISBN: 978-1-930934-19-1

Sullivan, B., & Kaszynski, A. (2019). PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). *Journal of Open Source Software*, *4*(37), 1450. https://doi.org/10.21105/joss.01450

The Qt Company. (2026). *PySide6: Official python bindings for Qt*. https://doc.qt.io/qtforpython/

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2